

函数用法和底层分析

- 1 函数是可重用的程序代码块。
- 2 函数的作用，不仅可以实现代码的复用，更能实现代码的一致性。一致性指的是，只要修改函数的代码，则所有调用该函数的地方都能得到体现。



在编写函数时，函数体中的代码写法和我们前面讲述的基本一致，只是对代码实现了封装，并增加了函数调用、传递参数、返回计算结果等内容。

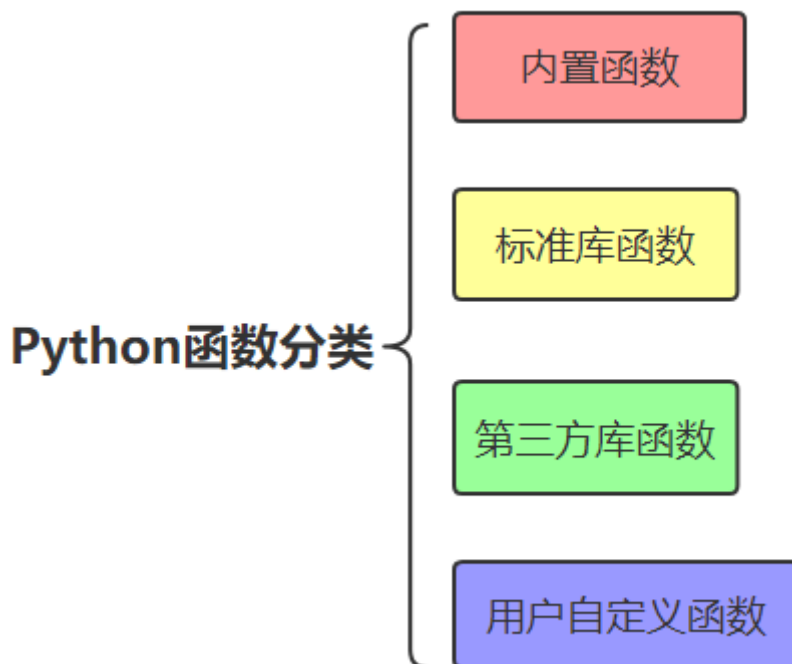
△ 为了让大家更容易理解，掌握的更深刻。我们也要深入内存底层进行分析。绝大多数语言内存底层都是高度相似的，这样大家掌握了这些内容也便于以后学习其他语言。

函数简介

函数(function)的基本概念

- 1 一个程序由一个一个的任务组成；函数就是代表一个任务或者一个功能(function)。
- 2 函数是代码复用的通用机制

Python函数的分类



Python函数分为如下几类：

① 内置函数

我们前面使用的 `str()`、`list()`、`len()` 等这些都是内置函数，我们可以拿来直接使用。

② 标准库函数

我们可以通过 `import` 语句导入库，然后使用其中定义的函数

③ 第三方库函数

Python社区也提供了很多高质量的库。下载安装这些库后，也是通过 `import` 语句导入，然后可以使用这些第三方库的函数

④ 用户自定义函数

用户自己定义的函数，显然也是开发中适应用户自身需求定义的函数。今天我们学习的就是如何自定义函数。

函数的定义和调用



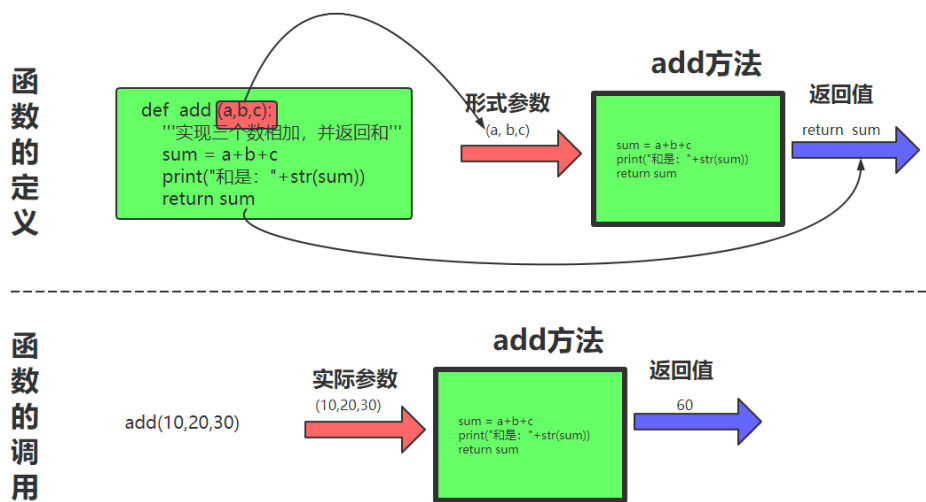
核心要点

Python中，定义函数的语法如下：

```
1 def 函数名 ([参数列表]) :  
2     '''文档字符串'''  
3     函数体/若干语句
```

简单定义一个函数：

```
1 def add(a,b,c):  
2     '''完成三个数的加法，并返回他们的和'''  
3     sum = a+b+c  
4     print("{0}、{1}、{2}三个数的和是：  
5         {3}".format(a,b,c,sum))  
6     return sum  
7 add(10,20,30)  
8 add(30,40,50)
```



要点:

- ① 我们使用 `def` 来定义函数，然后就是一个空格和函数名称；
 - Python执行 `def` 时，会创建一个函数对象，并绑定到函数名变量上。
- ② 参数列表
 - 圆括号内是形式参数列表，有多个参数则使用逗号隔开
 - 定义时的形式参数不需要声明类型，也不需要指定函数返回值类型
 - 调用时的实际参数必须与形参列表一一对应
- ③ `return` 返回值
 - 如果函数体中包含 `return` 语句，则结束函数执行并返回值；
 - 如果函数体中不包含 `return` 语句，则返回 `None` 值。
- ④ 调用函数之前，必须要先定义函数，即先调用 `def` 创建函数对象
 - 内置函数对象会自动创建
 - 标准库和第三方库函数，通过 `import` 导入模块时，会执行模块中的 `def` 语句

实时效果反馈

1. 如下关于函数的说法，错误的是：

- A 一个程序由一个个任务组成；函数就是代表一个任务或者一个功能。
- B 函数是代码复用的通用机制
- C Python执行 `def` 时，不会创建一个函数对象

D 如果函数体中包含return语句，则结束函数执行并返回值

答案

1=>C

形参和实参



形参和实参的要点：

- 圆括号内是形式参数列表，有多个参数则使用逗号隔开
- **定义时的形式参数**不需要声明类型，也不需要指定函数返回值类型
- **调用时的实际参数**必须与形参列表一一对应

【操作】 定义一个函数，实现两个数的比较，并返回较大的值

```
1 def printMax(a,b):  
2     '''实现两个数的比较，并返回较大的值'''  
3     if a>b:  
4         print(a,'较大值')  
5         return a  
6     else:  
7         print(b,'较大值')  
8         return b  
9  
10  
11 printMax(10,20)  
12 printMax(30,5)
```

执行结果：

20 较大值

30 较大值

上面的 `printMax` 函数中，在定义时写的 `printMax(a,b)`。`a` 和 `b` 称为“形式参数”，简称“形参”。也就是说，形式参数是在定义函数时使用的。形式参数的命名要符合“标识符”命名规则。

在调用函数时，传递的参数称为“实际参数”，简称“实参”。上面代码中，`printMax(10,20)`，`10` 和 `20` 就是实际参数。

文档字符串(函数的注释)

好的代码 是易懂的代码



程序的可读性最重要，一般建议在函数体开始的部分附上函数定义说明，这就是“文档字符串”，也有人成为“函数的注释”。我们通过三个单引号或者三个双引号来实现，中间可以加入多行文字进行说明。

【操作】测试文档字符串的使用

```

1  def print_star(n):
2      '''
3      根据传入的n，打印多个星号
4      :param n: 传入的数字
5      :return:  n个星号拼接的字符串
6      '''
7      s = "*" * n
8      print(s)
9      return s
10
11 help(print_star)
12 print(print_star.__doc__)

```

我们调用 `help(函数名)` 可打印输出函数的文档字符串。结果如下：

Help on function print_star in module **main**:

```
print_star(n)
```

根据传入的n，打印多个星号

我们也可以通过 `函数名.__doc__` 直接获取到函数的文档字符串，自己进行打印。

```
1 print(print_star.__doc__)
```

实时效果反馈

1. 如下关于形式参数和实际参数的说法，错误的是：

- A** 函数定义时，参数是形式参数。
- B** 定义时的形式参数不需要声明类型，也不需要指定函数返回值类型
- C** 调用时的实际参数必须与形参列表一一对应
- D** 函数调用时，参数是形式参数。

答案

1=>D

返回值



`return` 返回值要点:

- 1 如果函数体中包含 `return` 语句，则结束函数执行并返回值
- 2 如果函数体中不包含 `return` 语句，则返回 `None` 值
- 3 要返回多个值，使用列表、元组、字典、集合将多个值“存起来”即可

【操作】定义一个打印n个星号的无返回值的函数

```
1 def print_star(n):  
2     print("*"*n)  
3  
4 print_star(5)
```

【操作】定义一个返回两个数平均值的函数

```
1 def my_avg(a,b):  
2     return (a+b)/2  
3  
4 #如下是函数的调用  
5 c = my_avg(20,30)  
6 print(c)
```

【操作】返回一个列表

```
1 def printShape(n):  
2     s1 = "#" * n  
3     s2 = "$" * n  
4     return [s1,s2]  
5  
6  
7 s = printShape(5)  
8 print(s)
```

实时效果反馈

1. 如下关于函数的返回值的说法，错误的是：

- ☐ A 如果函数体中包含 `return` 语句，则结束函数执行并返回值
- ☐ B 如果函数体中不包含 `return` 语句，则返回 `None` 值
- ☐ C 要返回多个返回值，使用列表、元组、字典、集合将多个值“存起来”即可
- ☐ D 如果函数体中不包含 `return` 语句，则不返回任何值

答案

1=>D

函数也是对象，内存底层分析



Python中，“一切都是对象”。实际上，执行 `def` 定义函数后，系统就创建了相应的函数对象。我们执行如下程序，然后进行解释：

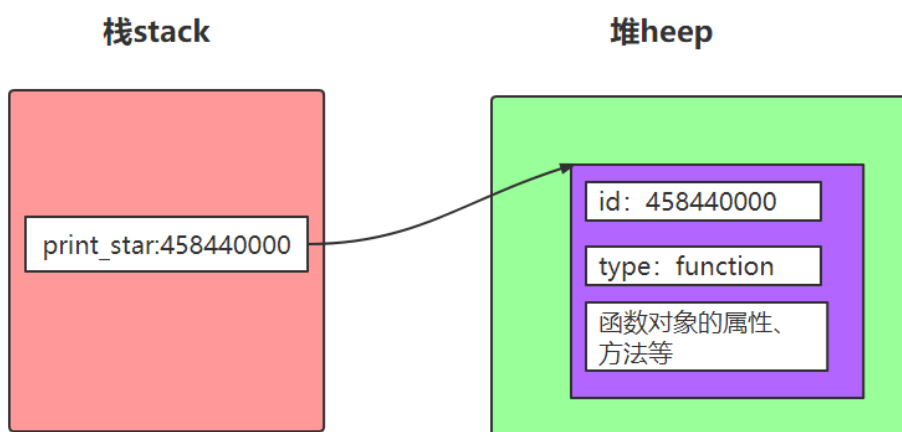
```
1 def print_star(n):  
2     print("*"*n)  
3  
4 print(print_star)  
5 print(id(print_star))  
6  
7 c = print_star  
8  
9 c(3)
```

执行结果：

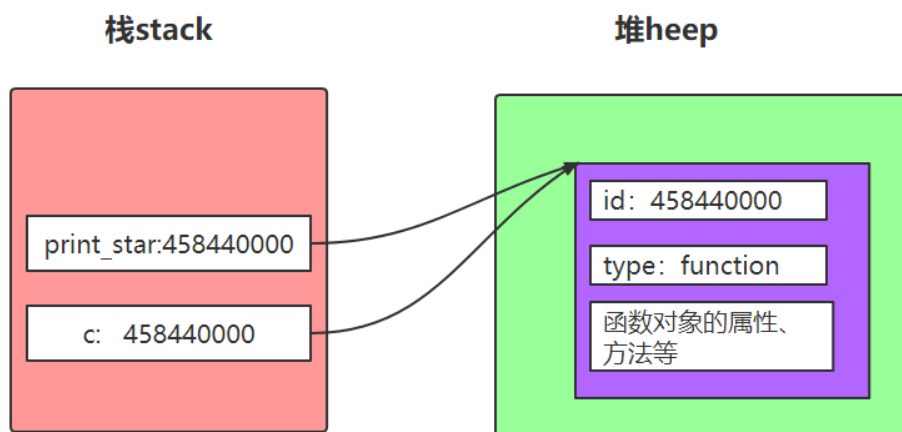
```
<function print_star at 0x0000000002BB8620>
```

```
45844000
```

上面代码执行 `def` 时，系统中会创建函数对象，并通过 `print_star` 这个变量进行引用：



我们执行 `c=print_star` 后，显然将 `print_star` 变量的值赋给了变量 `c`，内存图变成了：



- 1 显然，我们可以看出变量 `c` 和 `print_star` 都是指向了同一个函数对象。因此，执行 `c(3)` 和执行 `print_star(3)` 的效果是完全一致的。
- 2 Python中，圆括号意味着调用函数。在没有圆括号的情况下，Python会把函数当做普通对象。

与此核心原理类似，我们也可以做如下操作：

```
1 zhengshu = int
2 zhengshu("234")
```

显然，我们将内置函数对象 `int()` 赋值给了变量 `zhengshu`，这样 `zhengshu` 和 `int` 都是指向了同一个内置函数对象。

⚠当然，此处仅限于原理性讲解，实际开发中没必要这么做。

实时效果反馈

1. 如下代码，说法错误的是：

```
1 def print_star(n):
2     print("*"*n)
3
4 c = print_star
5 c(3)
```

- ☒ A 会有一个函数对象
- ☐ B `c` 和 `print_star` 变量指向了两个独立的函数对象
- ☐ C `c` 和 `print_star` 变量指向了同一个函数对象
- ☐ D `print_star` 变量指向了函数对象

答案

1=>B

变量的作用域(全局变量和局部变量)

变量起作用的范围称为变量的作用域，不同作用域内同名变量之间互不影响。变量分为：全局变量、局部变量。



全局变量：

- ① 在函数和类定义之外声明的变量。作用域为定义的模块，从定义位置开始直到模块结束。
- ② 全局变量降低了函数的通用性和可读性。应尽量避免全局变量的使用。
- ③ 要在函数内改变全局变量的值，使用 `global` 声明一下

局部变量：

- ① 在函数体中（包含形式参数）声明的变量。
- ② 局部变量的引用比全局变量快，优先考虑使用
- ③ 如果局部变量和全局变量同名，则在函数内隐藏全局变量，只使用同名的局部变量

【操作】全局变量的作用域测试

```

1  a = 100                #全局变量
2  def f1():
3      global a           #如果要在函数内改变全局变量的值，增
                           加global关键字声明
4      print(a)           #打印全局变量a的值
5      a = 300
6
7  f1()
8  f1()
9  print(a)
```

执行结果：

100

300

【操作】全局变量和局部变量同名测试

```

1 a=100
2 def f1():
3     a = 3      #同名的局部变量
4     print(a)
5
6 f1()
7 print(a)      #a仍然是100，没有变化

```

执行结果：

3

100

【操作】输出局部变量和全局变量

```

1 a = 100
2
3 def f1(a,b,c):
4     print(a,b,c)
5     print(locals())      #打印输出的局部变
    量
6     print("#"*20)
7     print(globals())     #打印输出的全局变
    量
8
9 f1(2,3,4)

```

输出结果：

```

1 2 3 4
2
3 {'c': 4, 'b': 3, 'a': 2}
4
5 #####
6
7 {'__name__': '__main__', '__doc__': None,
  '__package__': None, '__loader__': <class
  '_frozen_importlib.BuiltinImporter'>,
  '__spec__': None, '__annotations__': {},
  '__builtins__': <module 'builtins' (built-
  in)>, '__file__':
  'E:\\PythonExec\\if_test01.py', 'a': 100,
  'f1': <function f1 at 0x0000000002BB8620>}

```

实时效果反馈

1. 如下关于全局变量和局部变量的说法，错误的是：

- A** 全局变量：在函数和类定义之外声明的变量。作用域从定义位置开始直到模块结束
- B** 局部变量：在函数体中（包含形式参数）声明的变量
- C** 如果局部变量和全局变量同名，则在函数内隐藏全局变量，只使用同名的局部变量
- D** 局部变量的引用和全局变量一样快

答案

1=>D

局部变量和全局变量效率测试

局部变量的查询和访问速度比全局变量快，优先考虑使用，尤其是在循环的时候。



在特别强调效率的地方或者循环次数较多的地方，可以通过将全局变量转为局部变量提高运行速度。

【操作】测试局部变量和全局变量效率

```
1  import time
2
3  a = 1000
4  def test01():
5      start = time.time()
6      global a
7      for i in range(100000000):
8          a += 1
9      end = time.time()
10     print("耗时{0}".format((end-start)))
11
```

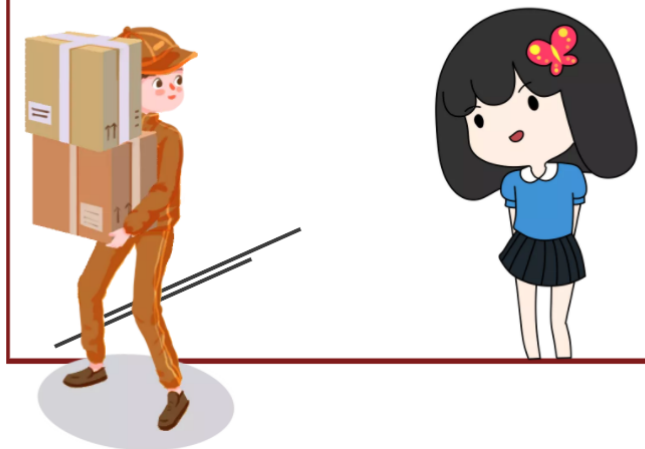
```
12 def test02():
13     c = 1000
14     start = time.time()
15     for i in range(100000000):
16         c += 1
17     end = time.time()
18     print("耗时{0}".format((end-start)))
19
20 test01()
21 test02()
22 print(globals())
```

运行结果：

```
耗时5.278882026672363
耗时3.6103720664978027
```

参数的传递

1. 函数的参数传递本质上就是：从实参到形参的赋值操作。
2. Python中“一切皆对象”，赋值操作都是“引用的赋值”。



函数的参数传递本质上就是：从实参到形参的赋值操作。Python中“一切皆对象”，所有的赋值操作都是“引用的赋值”。所以，Python中参数的传递都是“引用传递”，不是“值传递”。

具体操作时分为两类：

- ① 对“可变对象”进行“写操作”，直接作用于原对象本身。
- ② 对“不可变对象”进行“写操作”，会产生一个新的“对象空间”，并用新的值填充这块空间。

- 可变对象有：
字典、列表、集合、自定义的对象等
- 不可变对象有：
数字、字符串、元组、function等

传递可变对象的引用

传递参数是可变对象（例如：列表、字典、自定义的其他可变对象等），实际传递的还是对象的引用。在函数体中不创建新的对象拷贝，而是可以直接修改所传递的对象。

【操作】参数传递：传递可变对象的引用

```
1 b = [10, 20]
2 def f2(m):
3     print("m:", id(m))          #b和m是同一个对象
4     m.append(30)                #由于m是可变对象，不创建对象拷贝，直接修改这个对象
5
6 f2(b)
7 print("b:", id(b))
8 print(b)
```

执行结果：

```
m: 45765960
b: 45765960
[10, 20, 30]
```

实时效果反馈

1. 列表是可变对象，关于参数传递可变对象，说法错误的是：

```
1 b = [10, 20]
2 def f2(m):
3     print("m:", id(m))
4     m.append(30)
5
6 f2(b)
```

- A b和m是同一个对象
- B b和m是不同的对象
- C 对“可变对象”进行“写操作”，直接作用于原对象本身
- D `f2(b)` 把 `b` 传递给 `m` 的过程中，没有创建对象的拷贝

答案

1=>B

传递不可变对象的引用

传递参数是不可变对象（例如：`int`、`float`、字符串、元组、布尔值），实际传递的还是对象的引用。在“赋值操作”时，由于不可变对象无法修改，系统会新创建一个对象。

【操作】参数传递：传递不可变对象的引用

```
1 a = 100
2 def f1(n):
3     print("n:", id(n))           #传递进来的是a对象的地址
4     n = n+200                    #由于a是不可变对象，因此创建新的对象n
5     print("n:", id(n))           #n已经变成了新的对象
6     print(n)
7 f1(a)
8 print("a:", id(a))
```

执行结果：

```
n: 1663816464
n: 46608592
300
a: 1663816464
```

显然，通过 `id` 值我们可以看到 `n` 和 `a` 一开始是同一个对象。给 `n` 赋值后，`n` 是新的对象。

实时效果反馈

1. 数字是不可变对象，关于参数传递不可变对象并且要修改原对象，说法错误的是：

```

1 a = 100
2 def f1(n):
3     n = n+200
4
5 f1(a)

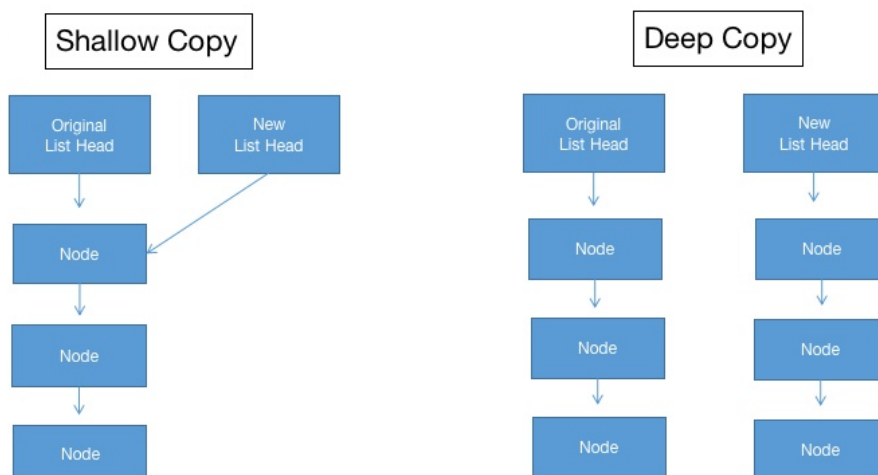
```

- A** 代码执行完后, `b` 和 `n` 是同一个对象
- B** 代码执行完后, `b` 和 `n` 是不同的对象
- C** 对“不可变对象”进行“写操作”, 创建新的对象
- D** 执行 `n=n+200` 时, 创建了新的对象

答案

1=>A

浅拷贝和深拷贝



为了更深入的了解参数传递的底层原理，我们需要讲解一下“浅拷贝和深拷贝”。我们可以使用内置函数：`copy` (浅拷贝)、`deepcopy` (深拷贝)。

- 1 **浅拷贝**：拷贝对象，但不拷贝子对象的内容，只是拷贝子对象的引用。
- 2 **深拷贝**：拷贝对象，并且会连子对象的内存也全部（递归）拷贝一份，对子对象的修改不会影响源对象

```
1 #测试浅拷贝和深拷贝
2 import copy
3
4 def testCopy():
5     '''测试浅拷贝'''
6     a = [10, 20, [5, 6]]
7     b = copy.copy(a)
8
9     print("a", a)
10    print("b", b)
11    b.append(30)
12    b[2].append(7)
13    print("浅拷贝.....")
14    print("a", a)
15    print("b", b)
16
17 def testDeepCopy():
18     '''测试深拷贝'''
19     a = [10, 20, [5, 6]]
20     b = copy.deepcopy(a)
21
22     print("a", a)
23     print("b", b)
```



```
24     b.append(30)
25     b[2].append(7)
26     print("浅拷贝.....")
27     print("a", a)
28     print("b", b)
29
30 testCopy()
31 print("*****")
32 testDeepCopy()
```

运行结果：

```
a [10, 20, [5, 6]]
b [10, 20, [5, 6]]
浅拷贝.....
a [10, 20, [5, 6, 7]]
b [10, 20, [5, 6, 7], 30]

a [10, 20, [5, 6]]
b [10, 20, [5, 6]]
深拷贝.....
a [10, 20, [5, 6]]
b [10, 20, [5, 6, 7], 30]
```

实时效果反馈

1. 如下关于浅拷贝和深拷贝，错误的是：

- A 浅拷贝使用函数: `copy` (浅拷贝)
- B 浅拷贝: 拷贝对象, 也全部拷贝子对象
- C 深拷贝: 拷贝对象, 也全部拷贝子对象
- D 深拷贝使用函数: `deepcopy` (深拷贝)

答案

1=>B

传递不可变对象包含的子对象是可变的情况

```
1 #传递不可变对象时。不可变对象里面包含的子对象是可变的。则方法内修改了这个可变对象，源对象也发生了变化。
2
3 a = (10,20,[5,6])
4 print("a:",id(a))
5
6 def test01(m):
7     print("m:",id(m))
8     m[2][0] = 888
9     print(m)
10    print("m:",id(m))
11
12 test01(a)
13 print(a)
```

运行结果:

a: 41611632

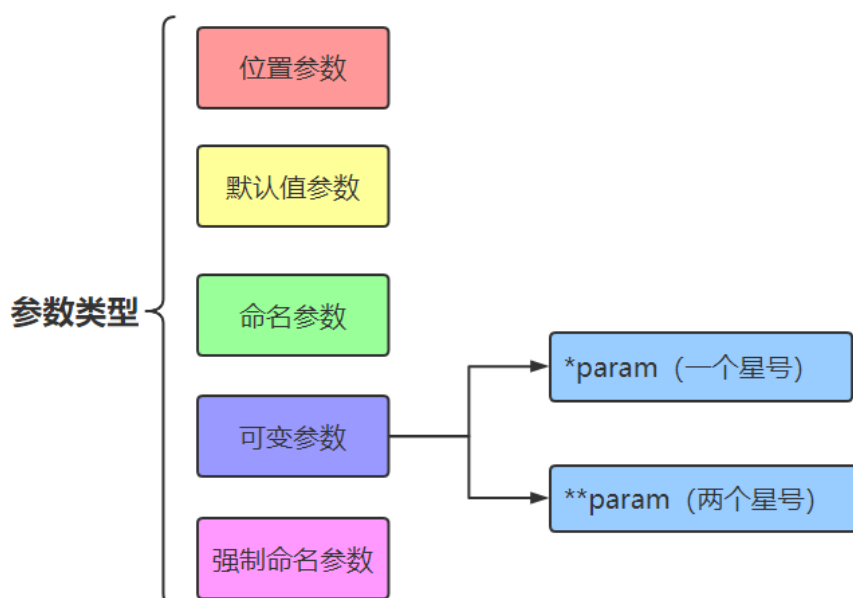
m: 41611632

(10, 20, [888, 6])

m: 41611632

(10, 20, [888, 6])

参数的几种类型



位置参数

函数调用时，实参默认按位置顺序传递，需要个数和形参匹配。按位置传递的参数，称为：“位置参数”。

【操作】测试位置参数

```
1 def f1(a,b,c):
2     print(a,b,c)
3
4 f1(2,3,4)
5 f1(2,3)      #报错，位置参数不匹配
```

执行结果：

```
1 2 3 4
2 Traceback (most recent call last):
3   File "E:\PythonExec\if_test01.py", line 5,
4   in <module>
5     f1(2,3)
5 TypeError: f1() missing 1 required positional
   argument: 'c'
```

默认值参数

我们可以为某些参数设置默认值，这样这些参数在传递时就是可选的。称为“默认值参数”。默认值参数放到位置参数后面。

【操作】测试默认值参数

```

1 def f1(a,b,c=10,d=20):    #默认值参数必须位于普通
    位置参数后面
2     print(a,b,c,d)
3
4 f1(8,9)
5 f1(8,9,19)
6 f1(8,9,19,29)

```

执行结果：

```

1 8 9 10 20
2 8 9 19 20
3 8 9 19 29

```

命名参数

我们也可以按照形参的名称传递参数，称为“命名参数”，也称“关键字参数”。

```

1 def f1(a,b,c):
2     print(a,b,c)
3
4 f1(8,9,19)           #位置参数
5 f1(c=10,a=20,b=30)  #命名参数

```

执行结果：

```

1 8 9 19
2 20 30 10

```

实时效果反馈

1. 如下函数定义后，调用时，错误的是：

```

1 def f1(a,b,c=10,d=20):    #默认值参数必须位于普通
    位置参数后面
2     print(a,b,c,d)

```

A `f1(5)`

B `f1(5,6)`

C `f1(5,6,7,8)`

D `f1(b=5,a=6,d=7,c=8)`

答案

1=>A

可变参数

可变参数指的是“可变数量的参数”。分两种情况：

- 1 `*param`（一个星号），将多个参数收集到一个“元组”对象中。
- 2 `**param`（两个星号），将多个参数收集到一个“字典”对象中。

【操作】测试可变参数处理（元组、字典两种方式）

```

1 def f1(a,b,*c):
2     print(a,b,c)
3
4 f1(8,9,19,20)
5
6
7 def f2(a,b,**c):
8     print(a,b,c)

```

```

9
10 f2(8,9,name='gaoqi',age=18)
11
12
13 def f3(a,b,*c,**d):
14     print(a,b,c,d)
15
16 f3(8,9,20,30,name='gaoqi',age=18)

```

执行结果：

```

1 8 9 (19, 20)
2 8 9 {'name': 'gaoqi', 'age': 18}
3 8 9 (20, 30) {'name': 'gaoqi', 'age': 18}

```

强制命名参数

在带星号的“可变参数”后面增加新的参数，必须在调用的时候“强制命名参数”。

```

1 def f1(*a,b,c):
2     print(a,b,c)
3
4
5 #f1(2,3,4)    #会报错。由于a是可变参数，将2,3,4全部
6               收集。造成b和c没有赋值。
7 f1(2,b=3,c=4)

```

执行结果：

```

1 (2,) 3 4

```

实时效果反馈

1. 如下关于可变参数的代码，调用时，错误的是：

```
1 def f1(a,b,*c):  
2     print(a,b,c)  
3  
4 def f2(a,b,**c):  
5     print(a,b,c)  
6  
7 def f3(a,b,*c,**d):  
8     print(a,b,c,d)
```

- A `f1(8,9,19,20)`
- B `f2(8,9,name='gaoqi',age=18)`
- C `f3(8,9,20,30,name='gaoqi',age=18)`
- D `f2(7)`

2. 如下是强制命名参数，调用时，错误的是：

```
1 def f1(*a,b,c):  
2     print(a,b,c)
```

- A `f1(2,3,4)`
- B `f1(2,b=3,c=4)`
- C `f1(2,c=3,b=4)`
- D `f1(20,c=3,b=4)`

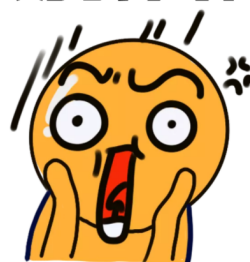
答案

1=>D 2=>A

lambda表达式和匿名函数

A a	B b	G g	D d	E e	Z z
Α α Alpha 阿尔法	Β β Beta 贝塔	Γ γ Gamma 伽玛	Δ δ Delta 德尔塔	Ε ε Epsilon 艾普西龙	Ζ ζ Zeta 捷塔
H h	Q q	I i	K k	L l	M m
Η η Eta 依塔	Θ θ Theta 西塔	Ι ι Iota 艾欧塔	Κ κ Kappa 卡帕	Λ λ Lambda 拉姆达	Μ μ Mu 缪
N n	X x	O o	P p	R r	S s
Ν ν Nu 努	Ξ ξ Xi 克西	Ο ο Omicron 欧麦克伦	Π π Pi 派	Ρ ρ Rho 柔	Σ σ Sigma 西格玛
T t	U u	F f	C c	Y y	W w
Τ τ Tau 套	Υ υ Upsilon 宇普西龙	Φ φ Phi 服艾	Χ χ Chi 器	Ψ ψ Psi 普赛	Ω ω Omega 欧米伽

Delta, 听起来
好吓人?



lambda 表达式可以用来声明匿名函数。**lambda** 函数是一种简单的、在同一行中定义函数的方法。**lambda** 函数实际生成了一个函数对象。

lambda 表达式只允许包含一个表达式，不能包含复杂语句，该表达式的计算结果就是函数的返回值。

lambda 表达式的基本语法如下：

```
1 | lambda arg1, arg2, arg3... : <表达式>
```

arg1 **arg2** **arg3** 为函数的参数。<表达式>相当于函数体。运算结果是：表达式的运算结果。

【操作】lambda表达式使用

```

1 f = lambda a,b,c:a+b+c
2 print(f)
3 print(f(2,3,4))
4
5 g = [lambda a:a*2,lambda b:b*3,lambda c:c*4]
6 print(g[0](6),g[1](7),g[2](8))

```

执行结果：

```

1 <function <lambda> at 0x0000000002BB8620>
2 9
3 12 21 32

```

实时效果反馈

1. 如下关于lambda表达式，错误的是：

```

1 f = lambda a,b,c:a+b+c
2 print(f(2,3,4))

```

- A lambda 函数是一种简单的、在同一行中定义函数的方法
- B lambda 函数实际生成了一个函数对象
- C lambda 表达式只允许包含一个表达式，该表达式的计算结果就是函数的返回值
- D lambda 函数 f 不会生成函数对象，但是可以通过 f(2,3,4) 调用

答案

1=>D

eval()函数

我是 eval() ,
小心点, 慎用我!



功能：将字符串 `str` 当成有效的表达式来求值并返回计算结果。

语法：`eval(source[, globals[, locals]]) -> value`

参数：

- 1 `source`：一个Python表达式或函数 `compile()` 返回的代码对象
- 2 `globals`：可选。必须是 `dictionary`
- 3 `locals`：可选。任意映射对象

```
1 #测试eval()函数
2
3 s = "print('abcde')"
4 eval(s)
5
6 a = 10
7 b = 20
8 c = eval("a+b")
9 print(c)
10
11 dict1 = dict(a=100,b=200)
12
13 d = eval("a+b",dict1)
14 print(d)
```

⚠ ⚠ ⚠ `eval`函数会将字符串当做语句来执行，因此会被注入安全隐患。比如：字符串中含有删除文件的语句。那就麻烦大了。因此，使用时，要慎重！！

实时效果反馈

1. 如下关于 `eval()` 函数，错误的是：

```
1 a = 10
2 b = 20
3 c = eval("a+b")
4 print(c)
```

A `eval(str)` 函数的作用：将字符串 `str` 当成有效的表达式来求值并返回计算结果。

- B** 上面代码的执行结果是：1020
- C** eval函数会将字符串当做语句来执行，因此会被注入，安全隐患
- D** 上面代码的执行结果是：30

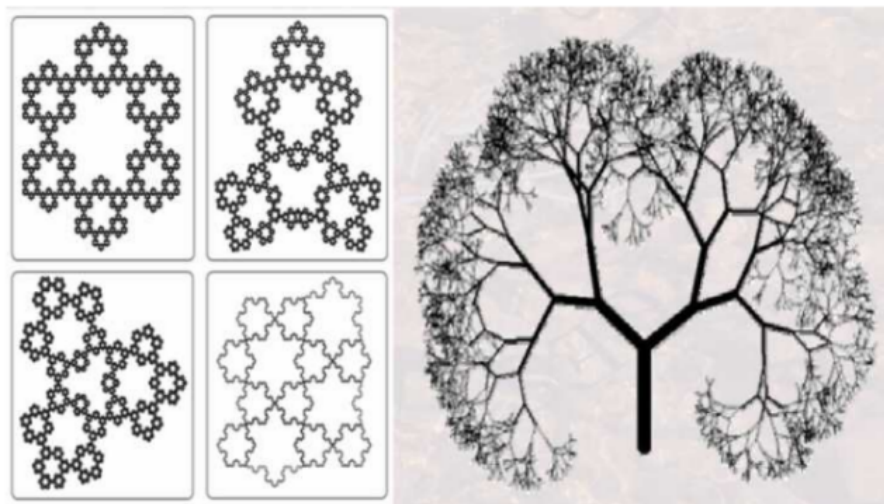
答案

1=>B

递归函数



- 1 递归(recursion)是一种常见的算法思路，在很多算法中都会用到。比如：深度优先搜索（DFS:Depth First Search）等。
- 2 递归的基本思想就是“自己调用自己”



递归函数指的是：自己调用自己的函数，在函数体内部直接或间接的自己调用自己。每个递归函数必须包含两个部分：

① 终止条件

表示递归什么时候结束。一般用于返回值，不再调用自己。

② 递归步骤

把第 n 步的值和第 $n-1$ 步相关联。

⚠ 递归函数由于会创建大量的函数对象、过量的消耗内存和运算能力。在处理大量数据时，谨慎使用。

如下是一个简单的递归程序：

```

1 def my_recursion(n):
2     print("start:" + str(n))
3     if n == 1:
4         print("recursion over!")
5     else:
6         my_recursion(n - 1)
7
8     print("end:" + str(n))
9
10 my_recursion(3)

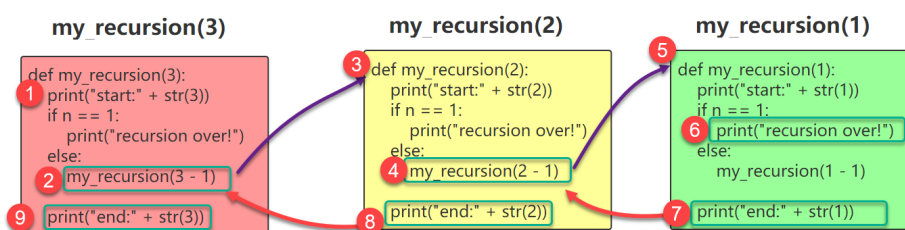
```

运算结果:

```

start:3
start:2
start:1
recursion over!
end:1
end:2
end:3

```



递归函数整个调用过程

实时效果反馈

1. 如下关于递归函数，错误的是：

- A 递归函数指的是：自己调用自己的函数
- B 深度优先搜索（DFS:Depth First Search）中使用了递归算法
- C 在处理大量数据时，谨慎使用
- D 递归函数可以不包含“终止条件”

答案

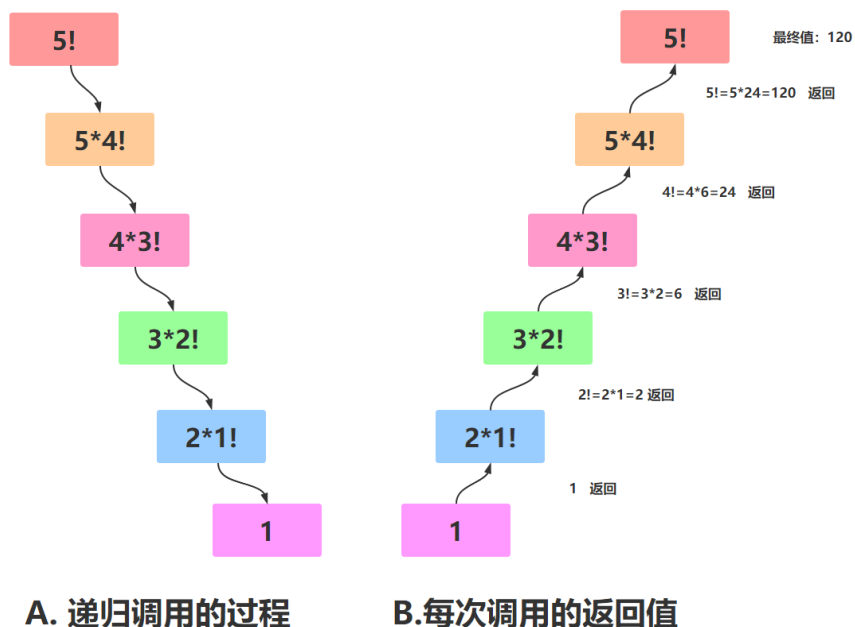
1=>D

【操作】 使用递归函数计算阶乘(factorial)

```
1 def factorial(n):  
2     if n==1:  
3         return 1  
4     return n*factorial(n-1)  
5  
6 print(factorial(5))
```

执行结果：

```
1 120
```

嵌套函数(内部函数)



嵌套函数：在函数内部定义的函数！

```
1 def outer():
2     print('outer running...')
3
4     def inner():
5         print('inner running...')
6
7     inner()
8
9 outer()
```

执行结果：

```
1 outer running...
2 inner running...
```

上面程序中，`inner()` 就是定义在 `outer()` 函数内部的函数。`inner()` 的定义和调用都在 `outer()` 函数内部。

一般在什么情况下使用嵌套函数？

- ① 封装 - 数据隐藏
外部无法访问“嵌套函数”。
- ② 贯彻 DRY(Don't Repeat Yourself) 原则
- ③ 嵌套函数，可以让我们在函数内部避免重复代码。
- ④ 闭包（后面会讲解）

【操作】使用嵌套函数避免重复代码

```
1 def printChineseName(name, familyName):  
2     print("{0} {1}".format(familyName, name))  
3  
4 def printEnglishName(name, familyName):  
5     print("{0} {1}".format(name, familyName))
```

使用1个函数代替上面的两个函数

```
1 def printName(isChinese, name, familyName):  
2     def inner_print(a, b):  
3         print("{0} {1}".format(a, b))  
4  
5     if isChinese:  
6         inner_print(familyName, name)  
7     else:  
8         inner_print(name, familyName)  
9  
10 printName(True, "小七", "高")  
11 printName(False, "George", "Bush")
```

实时效果反馈

1. 如下关于嵌套函数(内部函数), 错误的是:

```
1 def f1():  
2     print('f1 running...')  
3  
4     def f2():  
5         print('f2 running...')  
6  
7     f2()
```

- A 调用 `f1()`，则会执行：`f1()`、`f2()`
- B 调用 `f1()`，则只执行：`f1()`，不会执行 `f2()`
- C 嵌套函数：在函数内部定义的函数！
- D 嵌套函数，可以让我们在函数内部避免重复代码。

答案

1=>B

nonlocal关键字



`nonlocal` 用来在内部函数中，声明外层的局部变量。

global

函数内声明全局变量，然后才使用全局变量

【操作】 使用nonlocal声明外层局部变量

```
1  #测试nonlocal、global关键字的用法
2  a = 100
3
4  def outer():
5      b = 10
6
7      def inner():
8          nonlocal b          #声明外部函数的局部
                                变量
9          print("inner b:",b)
10         b = 20
11
12         global a             #声明全局变量
13         a = 1000
14
15     inner()
16     print("outer b:",b)
17
18 outer()
19 print("a: ",a)
```

实时效果反馈

1. 如下关于 **nonlocal** 和 **global** 的说法，错误的是：

A **nonlocal** 用来声明外层的局部变量。

B **global** 用来声明全局变量。

- C** 函数内用 `global` 声明全局变量，然后才可以使用全局变量
- D** 函数内不用 `nonlocal` 声明变量，也可以使用外层的局部变量

答案

1=>D

LEGB规则

Python在查找“名称”时，是按照LEGB规则查找的：



Local 指的就是函数或者类的方法内部

Enclosed 指的是嵌套函数（一个函数包裹另一个函数，闭包）

Global 指的是模块中的全局变量

Built in 指的是Python为自己保留的特殊名称

如果某个 `name` 映射在局部 `local` 命名空间中没有找到，接下来就会在闭包作用域 `enclosed` 进行搜索，如果闭包作用域也没有找到，Python 就会到全局 `global` 命名空间中进行查找，最后会在内建 `built-in` 命名空间搜索（如果一个名称在所有命名空间中都没有找到，就会产生一个 `NameError`）

```

1  #测试LEGB
2
3  s = "global"
4  def outer():
5      s = "outer"
6
7      def inner():
8          s = "inner"
9          print(s)
10
11     inner()
12
13 outer()

```

我们依次将几个 `s` 注释掉，观察控制台打印的内容，体会LEGB 的搜索顺序。

实时效果反馈

1. 如下代码，调用 `outer()` 函数，则会输出的是：

```
1 s = "global"
2 def outer():
3     s = "outer"
4
5     def inner():
6         s = "inner"
7         print(s)
8
9     inner()
10
11 outer()
```

A inner

B outer

C global

D 会报错

答案

1=>A

实操作业

- ① 定义一个函数实现反响输出一个整数。比如：输入3245，输出5432.
- ② 编写一个函数，计算下面的数列：

$$m(n) = \frac{1}{2} + \frac{2}{3} + \dots + \frac{n}{n+1}$$

- ③ 输入三角形三个顶点的坐标，若有效则计算三角形的面积；如坐标无效，则给出提示
- ④ 输入一个毫秒数，将该数字换算成小时数，分钟数、秒数
- ⑤ 使用海龟绘图。输入多个点，将这些点都两两相连